

Comprehensive Guide to Elixir, C, ESP32, and IoT Concepts: An Examination Preparation Resource

Introduction

This document provides a comprehensive overview of fundamental and advanced concepts across various programming paradigms and technologies, including the Elixir functional programming language, low-level C programming, ESP32 microcontroller development with FreeRTOS, and principles of IoT systems involving MQTT and RDF. The content is derived from a series of practical exercises designed to build a strong understanding of these domains. It aims to serve as a detailed resource for examination preparation, elucidating key principles, common patterns, and important trade-offs encountered in software and systems engineering. Each section delves into specific modules and quests, explaining the core concepts, how they are implemented, and their broader implications in designing robust, efficient, and scalable systems.

Section 1: Foundational Elixir Concepts and Idioms

This section explores the core features of the Elixir programming language, emphasizing its functional nature, concurrency model, and data manipulation capabilities.

1.1. Values, Tuples, and Pattern Matching

Elixir's approach to data handling is fundamentally rooted in immutability and the power of pattern matching. In Elixir, every valid code snippet evaluates to a value. Variable binding occurs not through traditional assignment, but via the match operator (`=`). This operator ensures that the structure on its left-hand side matches the value on its right-hand side. For instance, in the expression `r = d / 2`, the result of the division is bound to `r` if `r` is a new variable or if the pattern matches. This is distinct from imperative languages where `=` typically signifies mutable assignment. Elixir's variables, once bound, are immutable; a new binding for an existing variable name effectively creates a new variable (shadowing), rather than altering the original.

Tuples are fixed-size, ordered collections used to group a small, known number of items, often for returning multiple values from functions. For example, `{1, 2}` creates a tuple. Values can be extracted from tuples using pattern matching: `{x,y}={1,2}` binds `x` to 1 and `y` to 2. If a pattern match fails—for instance, attempting to match `{a, b}` with `{1, 2, 3}`—Elixir raises a `MatchError` because the

structures do not align. This structural validation is a key aspect of Elixir's safety.

A notable feature of pattern matching is the ability to ignore values using the underscore (`_`). In `{_, b} = {1, 2}`, the first element of the tuple is ignored, and `b` is bound to 2. This mechanism not only binds variables but also validates the shape of data, ensuring functions operate on correctly structured inputs. The immutability of data structures in Elixir, including tuples and the values bound through pattern matching, significantly enhances code reliability and predictability, especially in concurrent systems. While this can make some imperative programming patterns, like state mutation within loops, feel less direct, Elixir resolves this through recursion, higher-order functions, and process-based state management. The predictability gained from immutable data and explicit pattern matching often outweighs the initial adjustment from imperative paradigms, leading to more robust and maintainable systems.

1.2. Booleans, Conditionals (if/case), and Atoms

Elixir provides boolean values `true` and `false`, along with standard boolean operators `and`, `or`, and `not`, which operate strictly on these boolean values. These differ from `&&`, `||`, and `!` which can work on truthy/falsy values. Conditional logic is primarily handled by `if` and `case` constructs.

The `if` construct evaluates a condition and executes one of two blocks. It is treated as a macro that expands into a case statement and always returns a value. If an `else` clause is omitted and the condition is false, `if` returns `nil`. For simple binary decisions, `if` offers conciseness:

Elixir

```
if easy do
  IO.puts("It is easy")
else
  IO.puts("It is hard")
end
```

This code snippet, taken from an exercise determining cake-slicing difficulty based on an even or odd participant count, demonstrates a typical use of `if`.

The case construct provides more powerful pattern matching capabilities, allowing for branching based on multiple patterns, not just boolean conditions. This makes it more flexible than if, especially for structured data or multiple distinct values:

Elixir

```
case easy do
  true -> { :status, "easy" }
  false -> { :status, "hard" }
end
```

A key characteristic of case is that it executes only the first matching clause, and the order of clauses can be critical, especially when patterns overlap. More general patterns should typically follow more specific ones to avoid unintended behavior. The trade-off between if and case lies in simplicity versus expressiveness: if is cleaner for straightforward true/false logic, while case is better suited for complex pattern matching, such as with tagged tuples like { :ok, value } versus { :error, reason }. Misusing if in contexts where case is more appropriate can lead to fragile code.

Atoms are named constants whose value is their own name (e.g., :ok, :error, :status). They are frequently used in Elixir for tagging results or representing distinct states, enhancing code clarity and enabling more readable pattern matching. For example, { :ok, result } uses the atom :ok to signify a successful operation. This reliance on atoms reduces the need for "magic" strings or numbers, contributing to more robust and self-documenting code.

The use of atoms for tagging tuples, combined with case for pattern matching, forms a common and powerful idiom in Elixir for handling function return values and managing control flow based on structured outcomes. This explicitness in handling success and failure states aligns well with Elixir's philosophy of building fault-tolerant systems.

1.3. Anonymous Functions, Pipelining, and Data Generation

Anonymous functions in Elixir, declared using the fn... end syntax, are functions without a name, often used for short-lived or localized logic. They are invoked with a dot (.) notation, for example, c.(5), where c is a variable holding an anonymous function. An important detail is their strictness about arity; they must be called with the exact number of arguments they are defined to accept.

An example is defining a function to compute circumference:

Elixir

```
c = fn r -> 2 * :math.pi * r end
```

This function `c` can then be used to generate data.

Data generation often involves transforming collections, where `Enum.map/2` plays a core role. `Enum.map/2` applies a given function to each element of an enumerable (like a range or a list), returning a new list of the results. For instance, to generate circumference data for radii from 2 to 10:

Elixir

```
data = 2..10 |> Enum.map(fn x -> %{"radius" => x, "c(x)" => c.(x)} end)
```

Here, `Enum.map` iterates through the range `2..10`. For each `x`, an anonymous function creates a map containing the radius and its calculated circumference `c.(x)`. This list of maps is then suitable for visualization libraries like `VegaLite`.

The pipe operator (`|>`) significantly enhances the readability of such data transformations by chaining function calls. The result of the expression on the left of `|>` is passed as the first argument to the function call on its right. This creates a clear, top-down flow of data, reducing the need for nested calls. For plotting with `VegaLite`, this looks like:

Elixir

```
VL.new(width: 400, height: 300)  
|> VL.data_from_values(data)  
|> VL.mark(:point)  
|> VL.encode_field(:x, "radius", type: :quantitative, title: "Radius")
```

```
|> Vl.encode_field(:y, "c(x)", type: :quantitative, title: "Circumference")
```

Each step in the pipeline builds upon the previous, declaratively configuring the plot.

The trade-off between anonymous and named functions involves expressiveness and features. Anonymous functions are compact for inline logic but lack the multi-clause pattern matching and direct recursion capabilities of named functions (defined with `def` in modules). For complex branching or recursion, named functions are generally preferred. The combination of anonymous functions for simple transformations, `Enum.map` for collection processing, and the pipe operator for readable composition is a hallmark of idiomatic Elixir code, promoting clarity and functional design.

1.4. Lists, Higher-Order Functions, and Custom `map/filter`

Lists in Elixir are fundamental data structures, implemented as linked lists. They are processed using pattern matching, typically with the `[h|t]` syntax, which deconstructs a list into its head (`h`, the first element) and tail (`t`, the remaining elements). This structure lends itself well to recursive function definitions.

Higher-order functions, which accept other functions as arguments or return them, are central to Elixir's functional programming style. `Enum.map/2` and `Enum.filter/2` are prime examples, allowing abstract operations on collections. Reimplementing these functions, as explored in exercises, provides a deeper understanding of their mechanics and the power of recursion and pattern matching.

A custom `map/2` function can be defined with two clauses:

1. Base Case: `def map(, fun), do:`
 - If the list is empty, it returns an empty list.
2. Recursive Case: `def map([h|t], fun), do: [fun.(h) | map(t, fun)]`
 - It applies `fun` to the head `h`, and prepends this result to the recursive call of `map` on the tail `t`.

Similarly, a custom `filter/2` function can be implemented:

1. Base Case: `def filter(, _fun), do:`
 - An empty list results in an empty list. (The `_fun` indicates the function argument is not used in this clause).
2. Recursive Case:

Elixir

```
def filter([h|t], fun) do
  if fun.(h) do
```

```

    [h | filter(t, fun)]
  else
    filter(t, fun)
  end
end
end

```

- If `fun.(h)` (the predicate) returns `true`, the head `h` is included. Otherwise, it's skipped. The recursion continues on the tail `t`.

The use of multiple function clauses for pattern matching (e.g., `map(...)` vs. `map([h|t],...)`) is not mere syntactic sugar; it's a core mechanism for function dispatch in Elixir. This avoids explicit `if` statements for checking list emptiness and makes recursive definitions clearer and safer.

While writing custom versions of Enum functions is instructive, there are trade-offs. Built-in Enum functions are highly optimized (often implemented in C via NIFs for performance) and handle edge cases robustly. Custom implementations might be less efficient and potentially introduce bugs. Therefore, for production code, built-in functions are preferred unless highly specialized behavior is required. These exercises demonstrate the declarative and expressive nature of Elixir for list manipulation, underscoring the elegance of functional recursion.

1.5. Maps: Creation, Access, and Key Type Trade-offs

Maps in Elixir are versatile key-value stores, defined with the `%{}` syntax. They support fast lookups and can have keys of any Elixir term. Accessing map values can be done in several ways:

- `Map.get(map, key, default_value)`: Safely retrieves a value, returning `default_value` (or `nil` if not provided) if the key is absent. This is crucial for handling dynamically populated or potentially incomplete maps. Example: `Map.get(month_lengths, "feb")` or `Map.get(month_lengths, "mar", 0)`.
- Dot notation (`map.key`): A concise syntax for accessing values when keys are atoms. Example: `month_lengths = %{jan: 31, feb: 28}`, then `month_lengths.feb` yields 28.
- Pattern matching: `%{key => value_var} = map` or `%{atom_key: value_var} = map` extracts values. Example: `%{g: g_val} = %{r: 127, g: 255, b: 0}` binds `g_val` to 255.

Elixir maps allow any term as a key, including complex types like tuples (e.g., `%{ {:en, "jan"} => 31 }`). This offers expressiveness for encoding multi-dimensional relationships but can reduce readability if overused.

A significant consideration is the choice between atoms and strings for keys :

- **Atoms as Keys:**

- Pros: Offer slightly better performance and enable the clean `map.key` syntax.
- Cons: Atoms are not garbage-collected. Dynamically creating atoms from untrusted external input (e.g., user-submitted data) can lead to atom table exhaustion, a serious memory leak that can crash the Erlang VM. `map.foo` only works with atom keys; `m = %{"foo" => 1}`; `m.foo` will error, and `Map.get(m, :foo)` will return `nil`, which can be a subtle source of bugs if key types are mixed unintentionally.

- **Strings as Keys:**

- Pros: Safe to create dynamically, making them suitable for keys derived from external sources like JSON payloads.
- Cons: No dot notation access; lookups might be marginally slower.

The general guidance is to use atoms for fixed, internal keys known at compile time and strings for dynamic or external keys. Map updates, such as with `Map.put(map, key, value)`, are immutable operations; they return a new map with the change, leaving the original map untouched. Maps can also be nested, allowing for hierarchical data structures like configurations or calendars, accessed via chained lookups (e.g., `calendar`). This immutability and flexibility, combined with careful key type selection, make maps a cornerstone of data organization in Elixir.

1.6. Structs: Definition, Usage, and Comparison with Maps

Elixir structs provide a way to define map-like data structures with a fixed set of predefined keys, offering a degree of type safety and predictability over regular maps. A struct is essentially a map that contains a special `__struct__` key, whose value is the atom of the module name that defines the struct (e.g., `%Person{}` includes `__struct__: Person`). This enforces a defined structure, helping to prevent errors from typos in keys and improving code maintainability.

Structs are defined within a module using `defstruct`, which lists the allowed fields and their optional default values:

Elixir

```
defmodule Person do
  defstruct [:name, :age, nationality: :unknown, alive?: true]
```

end

Instances are created using the `%ModuleName{}` syntax: `p = %Person{name: "Ada", age: 42}`. Fields are accessed using dot notation (e.g., `p.age`) or pattern matching. Updates use a specific syntax that ensures the result is still a struct of the same type: `%Person{p | age: p.age + 1}`.

The primary trade-off between structs and maps lies in structure enforcement versus flexibility :

- **Structs:**
 - Pros: Provide compile-time checks for field existence (when using dot notation for access in the same module or with `@enforce_keys`). Ensure data consistency by defining a specific shape. Act as lightweight types, improving API robustness.
 - Cons: Less flexible than maps. Adding fields not defined in `defstruct` is possible using `Map.put/3`, but the result, while still technically a map, may not be treated as a "strict struct" by all parts of the system or by tools expecting the defined shape. This can lead to inconsistencies if code assumes a strict adherence to the `defstruct` definition.
- **Maps:**
 - Pros: Highly flexible, allowing arbitrary keys to be added or removed dynamically. Ideal for heterogeneous data or data whose structure is not known at compile time (e.g., parsing arbitrary JSON).
 - Cons: Lack the compile-time guarantees and explicit structure of structs, potentially leading to runtime errors if keys are misspelled or missing.

If the data's structure is known and consistency is paramount, structs are preferred. For dynamic or schemaless data, maps are more suitable. The choice often depends on whether the data is internal to the application (favoring structs for known entities) or comes from external, less predictable sources (favoring maps).

1.7. Concurrency: The Fork-Join Pattern with Tasks (Fibonacci Example)

Elixir's concurrency model, built upon lightweight processes, allows for efficient parallel execution of tasks. The fork-join pattern, demonstrated with the `Task` module for calculating Fibonacci numbers, is a common approach to parallelize computations. This pattern involves distributing work across multiple processes (`fork`) and then collecting their results (`join`).

The Fibonacci sequence, particularly its naive recursive implementation (`fib(n) =`

$\text{fib}(n-1) + \text{fib}(n-2)$), is computationally expensive due to its exponential time complexity. This makes it a good candidate to illustrate the benefits of concurrent execution.

A `Concurrent.map/2` function can be implemented to apply a function (like Fibonacci calculation) to each element of a list concurrently:

Elixir

```
defmodule Concurrent do
  def map(elements, fun) do
    # Fork phase: Spawn a task for each element
    tasks = Enum.map(elements, fn element ->
      Task.async(fn -> process(element, fun) end)
    end)

    # Join phase: Await results from each task
    Enum.map(tasks, fn task -> Task.await(task, :infinity) end)
  end

  # Helper function executed by each task
  defp process(element, fun) do
    fun.(element)
  end
end
```

Fork Phase: `Task.async(fn ->... end)` spawns a new lightweight process for each element in the elements list. Each process independently executes the fun (e.g., Fibonacci calculation) on its assigned element. `Task.async/1` returns a Task struct, which is a reference to the spawned asynchronous operation.

Join Phase: `Task.await(task, :infinity)` waits for the specified task to complete and returns its result. The `:infinity` timeout means it will wait indefinitely. This phase collects results from all spawned tasks.

The key trade-off here is parallelism versus overhead. While spawning many tasks can leverage multi-core processors for CPU-bound operations, each task incurs some overhead for creation, scheduling, and message passing (for results). For very small computations or small datasets, this overhead might negate the benefits of parallelism. However, for sufficiently large or complex independent computations like

the recursive Fibonacci, the gains from parallel execution on multi-core systems are substantial. It's important to understand that Elixir's concurrency enables parallel execution if hardware permits (multiple cores), but fundamentally provides a way to structure programs as many independent, non-blocking operations.

1.8. Advanced Concurrency: GenServer and Registry (Bakery Example)

For more complex stateful concurrent applications, Elixir's OTP (Open Telecom Platform) provides powerful abstractions like GenServer and Registry. A GenServer (Generic Server) is a behavior module for implementing the server logic in a client-server relation. It encapsulates state, executes code asynchronously, and can handle synchronous (call) and asynchronous (cast) messages. The Registry provides a way to register and look up named processes, facilitating dynamic discovery and interaction between processes.

The bakery example illustrates a multi-agent system with Bakers, Clerks, and Customers, each potentially modeled as a GenServer.

- **GenServer for Actors:** Each entity (Baker, Clerk, Customer) is a GenServer process, managing its own internal state and behavior. For instance, a Clerk GenServer might maintain a list of orders and interact with Baker GenServers.
- **Registry for Service Discovery:**
 - BakerRegistry: Bakers could register themselves here, perhaps by the product they bake. Clerks can then query this registry to find an available baker for a specific product.
 - ClerkRegistry: Clerks register themselves, allowing Customers to find an available clerk to place an order. This use of registries decouples the components; a Customer doesn't need to know the specific Process ID (PID) of a Clerk, only that a clerk service is available via the registry.

The interaction flow might be:

1. A Customer (GenServer) wants to order a cake. It looks up a Clerk in the ClerkRegistry.
2. The Customer sends an order message (e.g., GenServer.cast) to the chosen Clerk's PID.
3. The Clerk (GenServer) receives the order. It might then look up an available Baker for that cake type in the BakerRegistry.
4. The Clerk forwards the order to the Baker (GenServer).
5. The Baker processes the order and might send a confirmation back.

This architecture, using GenServers for stateful actors and Registries for dynamic

routing, creates a scalable and fault-tolerant system. The introduction of an intermediary like a Clerk decouples customers from bakers, allowing for load distribution and flexibility (e.g., adding new bakers without customers needing to change). While this layered approach adds some implementation complexity, it significantly enhances modularity, scalability, and resilience, which are key tenets of OTP design. The communication is via message passing between isolated processes, adhering to the actor model.

Section 2: Livebook and Elixir Ecosystem

This section covers tools and resources relevant to learning and working with Elixir, specifically focusing on Livebook and documentation access.

2.1. Understanding LiveMD Format

LiveMD files, the format used by Livebook, are an extension of Markdown (.md) files.¹ A key characteristic is that all valid LiveMD files are also valid Markdown files. This ensures that LiveMD notebooks remain human-readable and can be edited or viewed with any standard Markdown editor. Furthermore, this compatibility makes them friendly for version control systems like Git, where changes can be easily tracked using standard diff tools.

Livebook enhances Markdown by embedding specific annotations, typically as JSON-like objects within HTML comments (e.g., ``). These annotations are ignored by standard Markdown renderers but are interpreted by Livebook to enable interactive features. These features include:

- Execution of Elixir code blocks (usually fenced with ``elixir).
- Rendering of outputs from code execution, such as text, plots (e.g., via VegaLite), and user interface elements.
- Interactivity through elements like sliders, input fields, and other dynamic components.

The design choice of basing LiveMD on Markdown with comment-style annotations allows for a graceful blend of static documentation and dynamic, executable content. The trade-off is that the dynamic and interactive behaviors are only realized within the Livebook environment, but the core content remains accessible and portable as plain Markdown.

2.2. Information Resources for Elixir and Livebook

A variety of resources are available for learning Elixir and utilizing Livebook effectively.

The `documentation.livemd` file, referenced in the exercises, points to several key sources ¹:

- **Livcoding Videos:** Simple livecoding videos and demonstrations of Livebook features can be found on the Livebook YouTube channel (youtube.com/@livebookdev).
- **Module Documentation (Hexdocs):** Documentation for Elixir modules, including built-in modules and third-party libraries, is primarily hosted on Hexdocs (hexdocs.pm). Within Livebook itself, hovering over code elements like module names or function calls often triggers tooltips with inline documentation for those specific elements. For example, hovering over `String.to_integer("42")` would show documentation for the `String` module and its `to_integer/1` function.
- **Textual Introduction to Language Concepts:** A comprehensive textual introduction to Elixir language concepts is available in the official "Getting Started Guide" on Hexdocs (hexdocs.pm/elixir/introduction.html). Additional learning materials, including books, guides, and community tutorials, can be found through the official Elixir language website (elixir-lang.org).

These resources provide a multi-faceted approach to learning, combining video tutorials, interactive examples within Livebook, and detailed textual documentation, catering to various learning preferences.

Section 3: C Programming Fundamentals

This section transitions to the C programming language, covering its compilation process, memory management through pointers, algorithmic problem-solving, and low-level memory interpretation techniques.

3.1. Compilation and Execution

The C programming language requires a compilation step to transform human-readable source code into machine-executable code. This process is fundamental to C development and was introduced in the "Hello, World" exercise.¹ The typical workflow involves:

1. **Writing the Source Code:** A C program begins with source code written in a text file, typically with a `.c` extension (e.g., `hello.c`). The "Hello, World" program includes the standard input/output library (`#include <stdio.h>`) for functions like `printf` and defines a `main` function (`int main(int argc, char* argv)`), which is the entry point for execution.¹
2. **Compilation:** The source code is compiled using a C compiler, commonly GCC (GNU Compiler Collection). The command `gcc hello.c -o hello` compiles `hello.c`

and creates an executable file named `hello`.¹ This single command typically invokes several stages:

- **Preprocessing:** Handles directives like `#include` (inserting header file content) and `#define` (macro expansion).
 - **Compiling:** Translates the preprocessed C code into assembly language.
 - **Assembling:** Converts the assembly language into machine code (object files).
 - **Linking:** Combines object files and necessary library code to produce the final executable file.
3. **Execution:** The compiled program is run from the terminal using its filename, often prefixed with `./` to indicate the current directory (e.g., `./hello`).¹
 4. **Output:** The program's output, such as the string `"Hello, World\n"` from `printf`, is displayed on the terminal.¹

The separation of compilation and execution is a hallmark of compiled languages like C. This contrasts with interpreted languages where code is executed more directly. While compilation adds an extra step, it allows for optimizations that can lead to highly performant executables. However, this also means that C programs are generally less portable at the binary level; an executable compiled for one architecture (e.g., x86-64) will not run on a different one (e.g., ARM) without recompilation. Managing different compilers and build flags for various platforms can add complexity to C development.

3.2. Pointer Arithmetic and Array Manipulation

C provides powerful, low-level memory access capabilities through pointers and pointer arithmetic. This allows for direct manipulation of memory addresses but also introduces significant responsibilities for the programmer regarding memory safety.¹ An exercise demonstrated simulating 1-based array indexing using a 0-indexed array by manipulating a pointer.¹

The core C concept is that pointer arithmetic is scaled by the size of the data type the pointer points to. If `p` is an `int*`, `p + 1` advances the pointer address by `sizeof(int)` bytes, not just one byte.¹ In the exercise `index1.c`, an array `int a` is initialized. A pointer `int* b` is set to `&a`, and then `b` is decremented: `b = b - 1;`. This makes `b` point to the memory location *before* the actual start of array `a`.¹ Consequently, accessing `b` effectively accesses `a`, `b` accesses `a`, and so on, achieving a 1-based indexing illusion.¹

While this technique might offer syntactic convenience for certain algorithms, it is fraught with risks:

- **Undefined Behavior:** Accessing memory outside the allocated bounds of an

array (like `b` in `index1.c`, which points before `a`) results in undefined behavior in C.¹ The program might crash, produce incorrect results, or seemingly work correctly, depending on the compiler and system state, making such bugs hard to detect. The value 32764 printed for `b` in the example is arbitrary garbage data from that memory location.¹

- **Memory Safety:** C does not perform automatic bounds checking for array or pointer accesses. It is the programmer's responsibility to ensure that all memory accesses are valid. Writing to an out-of-bounds location can corrupt unrelated data or program structures, leading to difficult-to-diagnose bugs or security vulnerabilities like buffer overflows.

The trade-off is clear: C grants fine-grained control over memory, which can be used for performance optimizations or low-level manipulations, but this comes at the cost of requiring extreme diligence from the programmer to maintain memory safety.¹

Unlike memory-safe languages, C does not protect against these errors, underscoring the need for careful coding, thorough testing, and the use of tools like static analyzers and memory debuggers.

3.3. Algorithmic Problem Solving: Dynamic Programming and Bitmasking

The Traveling Salesman Problem (TSP) is a classic NP-hard combinatorial optimization problem. An exercise demonstrated solving a 15-city TSP instance using dynamic programming (DP) with bitmasking in C.¹ This approach significantly reduces complexity compared to a brute-force factorial search.

The core idea is to represent subsets of visited cities using a bitmask. If there are N cities, a bitmask is an integer where the i -th bit is set if city i has been visited. The DP state is typically defined as `dp[mask][pos]`, representing the minimum cost to visit all cities in the mask, ending at city `pos`, having started at a fixed city (e.g., city 0).

The C implementation involved ¹:

- A 2D array `distances` storing inter-city distances.
- A DP table `dp` initialized to -1 (or `INT_MAX`) to store memoized results.
- A path array to reconstruct the optimal path.
- A recursive function `tsp(mask, pos)`:
 - **Base Case:** If all cities are visited (i.e., `mask == (1 << CITY_COUNT) - 1`), it returns the distance from `pos` back to the starting city (city 0).
 - **Memoization:** If `dp[mask][pos]` is not -1, the stored result is returned.
 - **Recursive Step:** It iterates through all cities `city`. If city is not in mask (i.e., `(mask & (1 << city)) == 0`), it calculates the cost of going from `pos` to `city` and

then recursively calls `tsp(mask | (1 << city), city)`. It takes the minimum over all such valid next cities.

- The result is stored in `dp[mask][pos]`, and the chosen `best_city` is stored in `path[mask][pos]`.

The time complexity of this solution is $O(N^2 \cdot 2^N)$, and the space complexity is $O(N \cdot 2^N)$ for the `dp` and `path` tables.¹ For $N=15$, this is feasible ($15^2 \cdot 2^{15} \approx 225 \cdot 32768 \approx 7.3 \times 10^6$ operations).

Bitmasking DP is an elegant and efficient technique for problems involving subsets or permutations on a small number of items. However, its exponential nature limits its direct applicability to larger N . This highlights a common constraint in algorithm design: the trade-off between finding an optimal solution and the practical computational resources required. For larger TSP instances, heuristics or approximation algorithms are typically employed because exact solutions become intractable. This exercise demonstrates a scenario where an exact algorithm is viable due to the small problem size but also implicitly teaches about the boundaries of such approaches.

3.4. Type Punning and Memory Interpretation

Type punning in C refers to the practice of interpreting a region of memory as different data types at different times.¹ An exercise ("The Final Test") demonstrated this by allocating an array of ints, populating it, and then reading from it as if it were an array of longs.¹

The code initializes int values where `values[i] = i`. Then, it iterates and prints `((long*)values)[i]`. Since `sizeof(long)` (typically 8 bytes on 64-bit systems) is usually twice `sizeof(int)` (typically 4 bytes), each access to `((long*)values)[i]` reads the memory corresponding to two consecutive ints (`values[2*i]` and `values[2*i+1]`) and interprets these 8 bytes as a single long value.¹

The output, such as `0:0` followed by `values + values as long: 4294967296`, is revealing. The `4294967296` is 2^{32} . This occurs because, on a little-endian system:

- `values` (which is 0) occupies the lower 4 bytes of the long.
- `values` (which is 1) occupies the upper 4 bytes of the long. Thus, the 64-bit long value is effectively `(values << 32) | values`, which is $(1 \cdot 2^{32}) + 0 = 4294967296$.¹ Similarly for other pairs.

This behavior is highly dependent on:

- **Endianness:** The byte order (little-endian vs. big-endian) of the system determines how the bytes of the two ints are combined into the long. The code is not portable across systems with different endianness.
- **Data Type Sizes:** It assumes specific sizes for int and long.
- **Strict Aliasing Rules:** Accessing memory via a pointer of an incompatible type (like reading an int location as a long) often violates C's strict aliasing rules. Compilers use these rules for optimization, and violating them can lead to undefined behavior. The compiler might make assumptions that are no longer valid, resulting in unexpected code generation or behavior.

While type punning can sometimes be used for low-level optimizations or interfacing directly with hardware registers, it is generally considered unsafe and non-portable. It breaks type safety and can lead to bugs that are extremely difficult to diagnose because they depend on specific compiler optimizations, system architecture, and memory layout. Modern C standards discourage direct pointer casting for type punning; using unions is a more standard-compliant, though still potentially tricky, way to achieve similar reinterpretation of memory. This exercise serves as a strong caution against "clever" low-level manipulations that can compromise program correctness and portability.

Section 4: ESP32 Microcontroller and FreeRTOS Exercises

This section delves into programming the ESP32 microcontroller, with a focus on interacting with its General-Purpose Input/Output (GPIO) pins, Analog-to-Digital Converter (ADC), and Universal Asynchronous Receiver-Transmitter (UART), all managed within the FreeRTOS real-time operating system environment.

4.1. GPIO Configuration and Control (Output)

Controlling General-Purpose Input/Output (GPIO) pins for digital output is a fundamental aspect of microcontroller programming, often demonstrated by blinking an LED. Exercises involved setting up a GPIO pin as an output and toggling its state periodically using a FreeRTOS task.¹

The ESP-IDF framework provides APIs for GPIO manipulation:

- `gpio_reset_pin(PIN)`: Resets the pin to its default state.
- `gpio_set_direction(PIN, GPIO_MODE_OUTPUT)`: Configures the specified PIN as an output pin.¹
- `gpio_set_level(PIN, state)`: Sets the output level of the pin to state (1 for high, 0 for low).¹

To create a blinking effect, a FreeRTOS task is typically employed:

C

```
#define DELAY (1000 / portTICK_PERIOD_MS) // 1 second delay
#define PIN (33) // Example GPIO pin

void TaskBlink(void *pvParameters) {
    uint8_t state = 1;
    while (1) {
        gpio_set_level(PIN, state);
        vTaskDelay(DELAY); // Non-blocking delay
        state = !state; // Toggle state
    }
}

void app_main(void) {
    gpio_reset_pin(PIN);
    gpio_set_direction(PIN, GPIO_MODE_OUTPUT);
    xTaskCreate(TaskBlink, "Blink", 4096, NULL, 1, NULL); // Create and start the task
}
```

In this structure, `xTaskCreate` spawns a new task that executes the `TaskBlink` function.¹ The `while(1)` loop within `TaskBlink` continuously toggles the LED state. Critically, `vTaskDelay(DELAY)` is used to pause the task. `vTaskDelay` is a FreeRTOS API call that yields control of the CPU, allowing other tasks (including lower-priority system tasks) to run. This is essential for cooperative multitasking and system stability. Using a busy-waiting loop for delay (e.g., a for loop iterating many times) would monopolize the CPU, potentially starving other tasks and leading to issues like watchdog timer resets.¹ The exercise also explored extending this to control multiple LEDs at different speeds, which could be achieved by passing parameters (like pin number and delay duration) to instances of the blink task.¹

The use of `vTaskDelay` is not merely for timing; it's a fundamental mechanism for enabling the FreeRTOS scheduler to manage concurrent tasks effectively. Without such yielding points, a high-priority task could prevent essential system operations, underscoring the importance of understanding RTOS scheduling principles in

embedded development.

4.2. UART Communication (Input/Output)

Universal Asynchronous Receiver-Transmitter (UART) communication is a standard method for serial data exchange between a microcontroller and other devices, such as a host computer for debugging or command input.¹ Exercises covered initializing the UART, reading string input, converting it to numerical types, and performing timed serial output.

UART initialization on the ESP32 involves:

1. Defining a `uart_config_t` structure with parameters like baud rate (e.g., 115200), data bits (e.g., `UART_DATA_8_BITS`), parity (e.g., `UART_PARITY_DISABLE`), and stop bits (e.g., `UART_STOP_BITS_1`).¹
2. Installing the UART driver using `uart_driver_install(UART_NUM_O, RX_BUF_SIZE, TX_BUF_SIZE,...)` for a specific UART port (e.g., `UART_NUM_O` for USB communication).¹
3. Applying the configuration using `uart_param_config(UART_NUM_O, &uart_config)`.¹

Reading string input typically involves:

- Using `uart_read_bytes(UART_NUM_O, (uint8_t*)buffer, buffer_len - 1, timeout_ticks)` to read data into a buffer.¹ Reading `buffer_len - 1` bytes leaves space for a null terminator.
- Null-terminating the received buffer (`input_buf[len] = '\0';`) to treat it as a C string.¹
- Parsing the string, for example, converting it to an integer using `atoi(input_buf)` from `stdlib.h`.¹

The `atoi` function is simple but lacks robust error handling; if the input string is not a valid number, `atoi` might return 0 or behave unpredictably.¹ For production code, more robust parsing functions like `strtol` (which provides error checking) are preferable.

Timed serial output, such as a countdown, can be achieved by formatting numbers into strings (e.g., using `snprintf`) and then writing them to UART using `uart_write_bytes(UART_NUM_O, out_string, strlen(out_string))`. `vTaskDelay` is used between outputs to control the timing.¹

Serial communication, while seemingly simple, forms a critical interface for many embedded systems. It's used for debugging output, receiving configuration

commands, and even inter-device communication. The raw byte-stream nature of UART means that the application layer must implement its own protocols for message framing (e.g., newline termination, fixed-length packets, or start/end markers) and data parsing. Without careful design of these aspects, UART-based interfaces can be prone to errors and difficult to integrate with. The exercises involving command-line interfaces over UART ¹ highlight the need for tokenization, argument conversion, and clear error reporting to create a usable system.

4.3. GPIO Input: Polling vs. Interrupts

Monitoring changes on GPIO input pins is essential for detecting external events like button presses or sensor triggers. Two common methods for this are polling and interrupts, each with distinct trade-offs.¹

Polling:

In the polling approach, a FreeRTOS task periodically reads the state of the GPIO pin using `gpio_get_level(GPIO_INPUT_PIN)`. If the current state differs from the previously recorded state, a change (rising or falling edge) is detected and reported.¹

- **Pros:** Simpler to implement and understand.
- **Cons:** Can be CPU-intensive if the polling frequency is high. May miss very short input pulses if the polling interval is too long. Less responsive compared to interrupts. A small delay (e.g., `vTaskDelay(pdMS_TO_TICKS(20))`) is often added in the polling loop to reduce CPU load and can also provide rudimentary debouncing for mechanical switches.¹

Interrupts:

The interrupt-based approach configures the GPIO pin to trigger an interrupt on a state change (e.g., `GPIO_INTR_ANYEDGE` for both rising and falling edges).¹

1. An Interrupt Service Routine (ISR) is registered for the GPIO pin using `gpio_install_isr_service` and `gpio_isr_handler_add`.¹
 2. The ISR itself (`gpio_isr_handler` in the example) should be very short and fast. Its primary role is often to signal a higher-priority task that an event has occurred. This is typically done by sending a message (e.g., the GPIO number that triggered the interrupt) to a FreeRTOS queue using `xQueueSendFromISR`.¹
 3. A separate FreeRTOS task (`gpio_interrupt_task`) waits to receive messages from this queue using `xQueueReceive`. Upon receiving a message, this task performs the actual processing, such as debouncing the signal, determining the edge type, and reporting the change.¹
- **Pros:** Highly responsive to input changes. CPU-efficient as code only runs when an event occurs.
 - **Cons:** More complex to implement. ISRs have strict limitations (e.g., cannot call

blocking FreeRTOS APIs, should not perform lengthy operations). Requires careful management of shared data between ISR context and task context, often necessitating queues or semaphores. Debouncing is crucial for noisy inputs like mechanical buttons to prevent multiple interrupt triggers from a single physical event.¹

The choice between polling and interrupts involves balancing simplicity against efficiency and responsiveness. For applications requiring timely reactions to external events without wasting CPU cycles, the interrupt-driven approach, despite its added complexity, is generally superior. The common pattern of a minimal ISR queuing an event for a dedicated handler task is a cornerstone of responsive embedded system design. It ensures that the ISR completes quickly, minimizing interrupt latency, while allowing more complex processing to occur in the context of a standard FreeRTOS task with access to the full RTOS API.

4.4. ADC for Analog Input and Sensor Data Processing

The Analog-to-Digital Converter (ADC) on the ESP32 allows the microcontroller to measure analog voltages, typically from sensors that output a voltage proportional to a physical quantity like temperature or light level.¹

ADC Initialization and Configuration:

1. **Unit Initialization:** An ADC unit (e.g., `ADC_UNIT_1`) is initialized using `adc_oneshot_unit_init_cfg_t` and `adc_oneshot_new_unit`. The "oneshot" mode allows for on-demand sampling.¹
2. **Channel Configuration:** Each ADC channel (corresponding to a specific GPIO pin) is configured using `adc_oneshot_chan_cfg_t`, setting parameters like attenuation (`atten`) and bitwidth (`bitwidth`). Attenuation (e.g., `ADC_ATTEN_DB_12`) extends the measurable voltage range (e.g., up to 3.3V for 12dB). The ESP32 typically offers 12-bit resolution, meaning raw ADC values range from 0 to 4095.¹

Sampling and Calibration:

A dedicated sample function often encapsulates the reading logic:

`adc_oneshot_read(adc_handle, channel, &raw_result)` reads the raw digital value.¹

Raw ADC values can be affected by various factors like reference voltage inaccuracies and inherent non-linearities of the ADC. Calibration is crucial for converting these raw values into accurate real-world units like millivolts.¹ The ESP-IDF provides calibration functions:

- `example_adc_calibration_init(...)` attempts to initialize a calibration scheme (e.g., line-fitting or curve-fitting, depending on eFuse values and hardware support).¹
- If calibration is successful and a handle is obtained, `adc_cali_raw_to_voltage(cali_handle, raw_result, &voltage_mv)` converts the raw

ADC value to a calibrated voltage in millivolts.¹ The availability and type of calibration can be hardware-dependent, as indicated by log messages like eFuse not burnt, skip software calibration.¹ Without proper calibration, sensor readings might have systematic errors, impacting the reliability of the IoT application.

Sensor-Specific Conversion:

Once a calibrated voltage is obtained, it's often converted to a physical unit using a sensor-specific formula. For the LMT86 temperature sensor, the relationship between voltage and temperature is linear but inverse.¹ The conversion formula typically involves a slope and an intercept value derived from the sensor's datasheet (e.g., Slope: -10.9 mV/°C, Intercept: 2103 mV at 0°C for LMT86).¹

Example conversion:

float temperature_c = (voltage_mv - LMT86_INTERCEPT_MV) / LMT86_SLOPE_MV_PER_C;.¹

Pipelined Processing with FreeRTOS:

For continuous monitoring, ADC sampling, conversion, and data transmission can be structured into a pipeline using FreeRTOS tasks and queues.¹ A TaskSample (producer) might perform ADC reading and initial conversion, sending results to a queue. A TaskTransmit (consumer) would then read from this queue and handle data logging or network transmission.¹ This decouples the stages, allowing them to operate concurrently and improving system responsiveness.

The ADC can also be used for less conventional purposes, such as generating pseudo-random numbers for a game by taking advantage of the noise inherent in analog readings.¹

The following table summarizes interaction methods for common ESP32 peripherals:

Table 1: ESP32 Peripheral Interaction Methods

Peripheral	Interaction Goal	Polling Approach (Pros/Cons/Example)	Interrupt Approach (Pros/Cons/Example)	FreeRTOS Integration (Tasks, Queues)
GPIO Output (LED)	Visual indication, status display	Pros: Simple. Cons: N/A for basic timed output. Example: gpio_set_level in a timed loop (TaskBlink with	Pros: N/A (typically not interrupt-driven for output). Cons: Overkill. Example: N/A.	xTaskCreate for periodic toggling (TaskBlink). Queues could be used if LED state is controlled by other

		vTaskDelay). ¹		tasks/events.
GPIO Input (Button)	Detect external digital events	Pros: Simpler to implement. Cons: CPU intensive if frequent, may miss short events. Example: gpio_polling_task checking gpio_get_level periodically. ¹	Pros: CPU efficient, highly responsive. Cons: More complex (ISR, debouncing). Example: gpio_isr_handler sending to gpio_interrupt_task via gpio_evt_queue. ¹	ISR sends event via xQueueSendFromISR; Handler task blocks on xQueueReceive. Debouncing often done in handler task.
UART (Serial I/O)	Debugging, command interface, data transfer	Pros: Simple for basic, blocking reads/writes. Cons: Can block tasks if not handled carefully. Example: uart_read_bytes with timeout for input in app_main loop. ¹	Pros: Efficient for event-driven reception. Cons: Adds complexity (UART event queue). Example: (Not detailed in snippets, but ESP-IDF supports UART events).	Tasks can block on uart_read_bytes. For more complex scenarios, UART events can be handled by a dedicated task processing data from a UART driver queue.
ADC (Analog Sensor)	Measure analog voltages, read sensors	Pros: Simple for periodic, non-critical sampling. Cons: Blocks sampling task during conversion if not structured well. Example: adc_oneshot_read within TaskSample with vTaskDelay. ¹	Pros: Useful for threshold-triggered sampling (continuous ADC mode). Cons: More setup for continuous ADC. Example: (Oneshot mode used in snippets, continuous ADC with interrupts is	TaskSample (producer) performs ADC read and xQueueSendToBack; TaskProcess/Transmit (consumer) xQueueReceive for pipelined processing. ¹

			an alternative).	
--	--	--	------------------	--

This systematic approach to peripheral interaction, leveraging FreeRTOS features, is key to building efficient and responsive embedded applications on the ESP32.

Section 5: ESP32 Networking Exercises

This section covers exercises related to establishing network connectivity for the ESP32, primarily focusing on Wi-Fi station mode and basic TCP client communication, eventually leading to MQTT client implementation.

5.1. Wi-Fi Station Mode Connection

Connecting an ESP32 to an existing Wi-Fi Access Point (AP) in station mode is a foundational step for most IoT applications. The process involves several stages, managed by the ESP-IDF Wi-Fi stack and event system.¹

1. Initialization:

- `esp_netif_init()`: Initializes the underlying TCP/IP network interface layer (LwIP).¹
- `esp_event_loop_create_default()`: Creates the default system event loop, which handles events from various components, including Wi-Fi and TCP/IP.¹
- `esp_netif_create_default_wifi_sta()`: Creates a default network interface instance for Wi-Fi station mode.¹
- `wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT()`: Gets the default Wi-Fi initialization configuration.
- `esp_wifi_init(&cfg)`: Initializes the Wi-Fi driver.¹

2. Configuration using Kconfig and menuconfig:

SSID and password for the target AP are not hardcoded. Instead, they are configured at build time using `idf.py menuconfig`.

- A `Kconfig.projbuild` file in the project's main directory defines configuration options like `CONFIG_ESP_WIFI_SSID` and `CONFIG_ESP_WIFI_PASSWORD` as menu items.¹
- Running `idf.py menuconfig` allows the user to input these credentials.
- The build system saves these values into the `sdkconfig` file and makes them available as C preprocessor macros (e.g., `CONFIG_ESP_WIFI_SSID`) in the source code.¹ This approach enhances security by keeping credentials out of version control and allows for easy reconfiguration for different networks.

3. Event Handling:

An event handler function is registered to respond to Wi-Fi and IP events:

`esp_event_handler_instance_register(WIFI_EVENT, ESP_EVENT_ANY_ID,`


```
&event_handler,...)
esp_event_handler_instance_register(IP_EVENT, IP_EVENT_STA_GOT_IP,
&event_handler,...).1
```

The event_handler typically manages:

- WIFI_EVENT_STA_START: When the Wi-Fi station starts, esp_wifi_connect() is called to initiate connection to the configured AP.¹
- WIFI_EVENT_STA_DISCONNECTED: If disconnection occurs, esp_wifi_connect() is called again to attempt reconnection, providing resilience.¹
- IP_EVENT_STA_GOT_IP: When the ESP32 successfully obtains an IP address from the AP (usually via DHCP), this event is triggered. The assigned IP address can then be logged or used to initiate further network operations.¹

4. **Setting Mode and Configuration, and Starting Wi-Fi:**

```
C
wifi_config_t wifi_config = {
    .sta = {
        .ssid = CONFIG_ESP_WIFI_SSID,
        .password = CONFIG_ESP_WIFI_PASSWORD,
        //... other STA settings like authmode
    },
};
ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config));
ESP_ERROR_CHECK(esp_wifi_start());
```

This sets the Wi-Fi mode to station, applies the configuration (including SSID and password from menuconfig), and starts the Wi-Fi driver.¹

5. **Modularization:**

The Wi-Fi connection logic is often encapsulated in a reusable component (e.g., hl_wifi.c and hl_wifi.h) with an initialization function like hl_wifi_init(). This separates networking concerns from the main application logic, improving code organization and maintainability.

The ESP-IDF's event loop system is central to this asynchronous process. Instead of polling for connection status, the application registers handlers and reacts to events as they occur. This non-blocking, event-driven model is efficient and allows the ESP32 to perform other tasks while network operations are in progress, which is critical for responsive embedded applications.

5.2. TCP Client Communication

Once the ESP32 has an IP address, it can act as a TCP client to connect to a remote server and exchange data. Exercises involved establishing a TCP socket connection and transmitting a counter or sensor data.¹

The architecture typically involves:

1. **Callback on IP Assignment:** A callback function (e.g., `connected_callback`) is invoked by the Wi-Fi module after the `IP_EVENT_STA_GOT_IP` event. This ensures that TCP operations are attempted only after network connectivity is established.¹ This callback often creates a FreeRTOS task to handle the TCP communication.
2. **Socket Address Preparation:**
A `sockaddr_in_t` structure holds the server's IP address and port. A helper function like `hl_wifi_make_addr(char* ip_str, uint16_t port)` can be used to populate this structure:

```
C
sockaddr_in_t hl_wifi_make_addr(char* ip_str, uint16_t port) {
    sockaddr_in_t addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET; // IPv4
    addr.sin_port = htons(port); // Convert port to network byte order
    addr.sin_addr.s_addr = inet_addr(ip_str); // Convert IP string to network byte order
    return addr;
}
```

This function uses `htons` (host to network short) for the port and `inet_addr` for the IP address.¹

3. **Socket Creation and Connection:**
A TCP socket is created using `socket(AF_INET, SOCK_STREAM, IPPROTO_IP)`. `AF_INET` specifies IPv4, `SOCK_STREAM` specifies TCP (a stream-oriented protocol), and `IPPROTO_IP` is the protocol.¹
The connection to the server is established using `connect(sock, (struct sockaddr*)&dest_addr, sizeof(dest_addr))`. Error handling is crucial for both `socket()` and `connect()` calls, checking return values and `errno`.¹
4. **Data Transmission:**
Data is sent using the `send(sock, buffer, length, flags)` function. A wrapper like `hl_wifi_tcp_tx(int sock, void* buffer, uint16_t length)` is useful because `send` might not transmit all requested bytes in a single call (partial send). The wrapper should loop until all bytes are sent or an error occurs ¹:

```
C
```

```

void hl_wifi_tcp_tx(int sock, void* buffer, uint16_t length) {
    uint16_t offset = 0;
    while (offset < length) {
        int sent_bytes = send(sock, (char*)buffer + offset, length - offset, 0);
        if (sent_bytes < 0) { /* handle error */ return; }
        offset += sent_bytes;
    }
}

```

5. FreeRTOS Task for Continuous Transmission:

A FreeRTOS task (e.g., TaskCount or TaskSample combined with TaskTransmit) is used for continuous data transmission.¹ This task typically:

- Formats data into a buffer (e.g., using sprintf).
- Calls the TCP transmit function.
- Uses vTaskDelay() to control the transmission rate (e.g., send data every second).

This modular approach, separating Wi-Fi connection (hl_wifi_init), address creation (hl_wifi_make_addr), TCP connection (hl_wifi_tcp_connect), and transmission (hl_wifi_tcp_tx), makes the networking code more manageable and reusable. Performing network operations within a dedicated FreeRTOS task prevents blocking of the main application or other critical system functions, as network calls can be inherently blocking or take a significant amount of time to complete. The use of a callback to trigger the start of these network tasks ensures that they only commence once the underlying Wi-Fi and IP layers are ready.

5.3. MQTT Client on ESP32 for Chat Application

The Message Queuing Telemetry Transport (MQTT) protocol is a lightweight publish-subscribe messaging protocol, well-suited for IoT applications due to its low overhead and ability to handle unreliable networks. An exercise involved implementing a simple chat application on the ESP32 using an MQTT client.¹

The ESP-IDF provides an MQTT client library (mqtt_client.h) for this purpose. The key steps are:

1. **Wi-Fi Connection:** First, the ESP32 must be connected to Wi-Fi and have an IP address, as established in previous exercises. A connected_callback from the Wi-Fi module signals readiness.¹
2. **MQTT Client Initialization:**
Inside the connected_callback (or a task started by it):

- An `esp_mqtt_client_config_t` structure is populated with the broker URL (e.g., `mqtt_cfg.broker.address.uri = "mqtt://broker.hivemq.com"`).¹
 - The MQTT client is initialized using `esp_mqtt_client_init(&mqtt_cfg)`, which returns a client handle (`esp_mqtt_client_handle_t`).¹
3. MQTT Event Handler Registration:
- An event handler function (`mqtt_event_handler`) is registered to process various MQTT events:
- ```
esp_mqtt_client_register_event(client_handle, ESP_EVENT_ANY_ID,
mqtt_event_handler, NULL).1
```
- This handler uses a switch statement on `event_id` to manage different events 1:
- `MQTT_EVENT_CONNECTED`: Triggered when the client successfully connects to the broker. Inside this case:
    - The client subscribes to the chat topic:
 

```
esp_mqtt_client_subscribe(client_handle, TOPIC, qos)
```

 (e.g., TOPIC defined as "org/sdu/course/iot/year/2024/chat/channel/42", qos typically 1 for chat).<sup>1</sup>
    - A FreeRTOS task (e.g., TaskChat) is created to handle publishing messages.<sup>1</sup>
  - `MQTT_EVENT_DISCONNECTED`: Logs disconnection.
  - `MQTT_EVENT_SUBSCRIBED`: Logs successful subscription.
  - `MQTT_EVENT_PUBLISHED`: Logs successful publication.
  - `MQTT_EVENT_DATA`: Triggered when a message is received on a subscribed topic. The payload (`event->data` of length `event->data_len`) is typically printed to the serial console for the chat application.<sup>1</sup>
  - `MQTT_EVENT_ERROR`: Logs errors.
4. Starting the MQTT Client:
- ```
esp_mqtt_client_start(client_handle)
```
- initiates the connection attempt to the broker.
- ¹
5. Publishing Messages (within TaskChat):
- The TaskChat FreeRTOS task periodically constructs a message (e.g., `sprintf(publish_buffer, "%s: %s\n", nick, line_buffer);`) and publishes it using `esp_mqtt_client_publish(client_handle, TOPIC, data, len, qos, retain)`.¹ A `vTaskDelay` controls the publishing frequency.

The MQTT client library itself typically runs its own internal FreeRTOS tasks to manage the network connection, protocol keep-alives, and asynchronous message handling. The `mqtt_event_handler` serves as the interface between these background MQTT operations and the application's logic. This event-driven architecture allows the ESP32 to efficiently handle concurrent MQTT publishing and subscribing without blocking

other operations, making it suitable for interactive applications like the chat room. The choice of QoS (Quality of Service) is important: QoS 0 (at most once) is fast but unreliable; QoS 1 (at least once) ensures delivery but allows duplicates; QoS 2 (exactly once) is most reliable but has higher overhead. For a chat application, QoS 1 is often a reasonable compromise.¹

Section 6: MQTT and Elixir Stream Processing Exercises

This section explores the use of MQTT for message transport in conjunction with Elixir for stream processing, covering concepts from basic broker setup to advanced data transformation pipelines using GenServers and Flow.

6.1. MQTT Broker Setup and Basic Pub/Sub

Setting up an MQTT broker is the first step in building MQTT-based applications. A lightweight broker like Mosquitto is often used for local development and testing.¹ The process involves:

1. **Installation:** Installing the MQTT broker software (e.g., `sudo apt-get install mosquitto mosquitto-clients` on Ubuntu) and command-line client tools (`mosquitto_pub` for publishing, `mosquitto_sub` for subscribing).¹
2. **Verification:** Testing the installation by running `mosquitto_sub -t mytopic` in one terminal to subscribe to "mytopic". In another terminal, `mosquitto_pub -t mytopic -m "hello, world"` publishes a message to "mytopic". The message "hello, world" should appear in the subscriber's terminal, confirming the broker is operational.¹

The MQTT broker acts as a central intermediary in the publish-subscribe pattern. Producers publish messages to specific "topics" on the broker, and consumers subscribe to topics they are interested in. The broker is responsible for filtering and routing messages from producers to all relevant subscribed consumers.¹ This decouples producers from consumers; they do not need direct knowledge of each other, only of the broker and the topic names. This loose coupling is a key advantage, enabling flexible and scalable distributed systems, particularly in IoT where devices and applications may join or leave the network dynamically.

6.2. Elixir Signal Generator (Producer GenServer)

To simulate IoT devices publishing data, an Elixir-based signal generator can be created using a GenServer for each simulated sensor. This Producer GenServer periodically reads predefined sample data and publishes it to a specific MQTT topic using a library like Tortoise.¹

The Producer GenServer typically has the following structure ¹:

- **init(args):** Initializes the GenServer's state, which includes the MQTT client ID, the target topic, a list of samples (e.g., `%{time: relative_time, value: sensor_value}`), and the current index into the samples list. It calls `schedule_next_sample` to start the publishing cycle.
- **handle_info(:publish_sample, state):** This callback is triggered by a message sent via `Process.send_after`. It performs the following:
 1. Retrieves the current sample value using `Enum.at(samples, index)`.
 2. Constructs a JSON message, often including the current system time (`:os.system_time(:milli_seconds)`) and the sample value.
 3. Publishes the message to the MQTT topic using `Tortoise.publish(client_id, topic, message, qos: 0)`.
 4. Calculates the next_index, looping back to the beginning of the samples list if the end is reached (`rem(index + 1, length(samples))`).
 5. Updates its state with the next_index.
 6. Calls `schedule_next_sample` again to schedule the next publication.
- **schedule_next_sample(state):** A helper function that calculates the delay until the next sample should be published. The delay is often derived from the relative time differences in the predefined samples list. It then uses `Process.send_after(self(), :publish_sample, delay)` to send a message to its own process after the calculated delay.

Concurrency is achieved by spawning one Producer GenServer process for each unique sensor stream (e.g., `room1/temp`, `room1/rhum`).¹ Each GenServer runs independently. The use of `Process.send_after` for timing is crucial: it sends an asynchronous message to the GenServer's own message queue. This means the GenServer doesn't block while "waiting"; it schedules a future event and remains available to process other messages (though in this simple producer, it mainly self-schedules).¹ This non-blocking, self-scheduling mechanism is highly efficient for managing periodic tasks within Elixir's actor model. For *n* sensor configurations, the system uses *n* Producer processes, plus one Tortoise MQTT client process and typically one Tortoise supervisor process.¹

6.3. Elixir Stream Transformation (Moving Average, Absolute Humidity)

Elixir GenServers can be used to consume MQTT messages, maintain state for individual data streams, perform transformations, and publish new, derived data streams back to MQTT. This was demonstrated with a moving average filter and an absolute humidity calculator.¹

Moving Average Filter 1:

A MovingAverage GenServer is created for each sensor stream (e.g., for each unique temperature sensor topic).

- **State:** It maintains a window (a list) of the most recent N samples (e.g., @window_size = 3).¹
- **Message Handling (handle_cast({:consume, {time, value}}, state)):**
 1. The new sample {time, value} is added to the window.
 2. The window is trimmed to keep only the last @window_size samples (new_window = ([{time, value}] ++ window) |> Enum.take(@window_size)).¹
 3. The average of the values in new_window is calculated.
 4. The calculated average (with the latest timestamp) is published to a new MQTT topic (e.g., dk/sdu/iot/2024/tester/mavg/room1/temp).
- **Dispatcher and Registry:** A Dispatcher GenServer subscribes to raw sensor data topics (e.g., dk/sdu/iot/2024/tester/siggen/+/+). When a message arrives, the Dispatcher uses a Registry (e.g., :mavg_registry) to find or dynamically start the appropriate MovingAverage GenServer for that specific source topic. It then casts the sample to that worker GenServer.¹ If an MQTT message is lost, the MovingAverage GenServer for that stream will simply calculate the average based on fewer samples in its window, demonstrating resilience. The length(new_window) dynamically adjusts the divisor.¹

Absolute Humidity Calculator 1:

An AbsoluteHumidity GenServer is created for each room.

- **State:** It maintains the latest temp and rhum (relative humidity) values received for that room, initially nil.¹
- **Message Handling (handle_cast({:update_temp,...}), handle_cast({:update_rhum,...})):** When a temperature or humidity update arrives, the GenServer updates the corresponding value in its state.
- **Calculation and Publication (maybe_calculate_and_publish):** After any update, this helper function checks if both temp and rhum values are present (not nil). If so, it calculates absolute humidity using the provided formula: $H_{abs} = 6.112 \cdot e^{\{(17.67 \cdot T)/(T+243.5)\}} \cdot H_{rel} \cdot (2.1674 / (273.15 + T))$.¹ The result is published to a distinct MQTT topic (e.g., prefix/abshum/room1).
- **Dispatcher and Registry:** Similar to the moving average, an AbsHumidityDispatcher subscribes to raw temperature and humidity topics. It parses the topic to identify the room and sensor_type ("temp" or "rhum"). It then uses a Registry (:abshum_registry) to find or create the AbsoluteHumidity GenServer for that room and casts the update message to it.¹

This dispatcher + dynamic worker GenServer (per key like room or topic) + Registry

pattern is a robust and scalable architecture in Elixir/OTP. It allows for concurrent processing of many independent data streams while maintaining isolated state for each. If one worker process encounters an error, it doesn't affect others, contributing to the system's overall fault tolerance. The dynamic creation of workers means resources are only allocated for active streams.

6.4. MQTT Topic Structure and System Latency

Effective MQTT topic design and an understanding of system latency are crucial for scalable and performant IoT systems.¹

MQTT Topic Structure:

A hierarchical topic structure is recommended, for example:

base_prefix/location/device_type/sensor_type[/unit].1

- base_prefix: (e.g., sdu/iot) for overall system segregation.
- location: (e.g., room1, laptop1, outdoor) for physical or logical grouping.
- device_type: (e.g., sensor, computer) for the source type.
- sensor_type: (e.g., temperature, uptime, humidity) for the measurement.
- unit (optional): (e.g., celsius, fahrenheit) for disambiguation.

This structure allows for:

- **Clear Differentiation:** sdu/iot/laptop1/computer/uptime is distinct from sdu/iot/room1/sensor/temperature/celsius.¹
- **Flexible Subscriptions:** Consumers can use wildcards.
sdu/iot/+ /sensor/temperature/celsius subscribes to all Celsius temperatures.
sdu/iot/room1/# subscribes to all data from room1.
- **Scalability:** Easily accommodates new sensor types (e.g., sdu/iot/room1/sensor/humidity/percent) or new locations (e.g., sdu/iot/outdoor/sensor/temperature/celsius) without altering the fundamental structure.¹

A well-designed topic hierarchy also plays a role in security and data governance. MQTT brokers can use Access Control Lists (ACLs) based on topic patterns to restrict publish/subscribe permissions for different clients. This prevents unauthorized data access or injection and helps manage data flow in complex systems with numerous participants.

System Latency:

End-to-end latency in an MQTT-based system is the sum of delays across various stages 1:

1. **T1-T0 (Sensor Acquisition to Publisher Ready):** Time for sensor reading and packaging by the publishing device.

2. **T2-T1 (Publisher Network Latency):** Transmission from publisher to MQTT broker.
3. **T3-T2 (Broker Processing Latency):** Broker's internal processing and routing.
4. **T4-T3 (Subscriber Network Latency):** Transmission from broker to a subscribing process (e.g., a data converter).
5. **T5-T4 (Subscriber Processing Latency):** Time for the subscriber to decode, process (e.g., convert Celsius to Fahrenheit), and prepare a new message.
6. **T6-T5 (Converted Message Network Latency):** Transmission of the processed message from the converter back to the broker, and then to a final consumer. ¹

Moving a processing component (e.g., the converter) to a remote location (like a friend's laptop over the internet) significantly increases network latencies (T2-T1, T4-T3, T6-T5 components), making them dominant factors in the overall delay. Local network latencies are typically in milliseconds, while internet latencies can be tens to hundreds of milliseconds and more variable (jitter).¹ This impacts real-time responsiveness and throughput. MQTT Quality of Service (QoS) levels also affect latency: QoS 0 is fastest (fire-and-forget), while QoS 1 and 2 add overhead for reliability guarantees.

6.5. Elixir Flow for Batch/Parallel Processing

Elixir's Flow library, built on GenStage, provides abstractions for building concurrent, distributed data processing pipelines, particularly suited for batch processing or transforming large collections of data in parallel.¹ This contrasts with the GenServer-based approach, which is more tailored for handling individual, asynchronous events with per-entity state.

The exercises demonstrated word counting using different approaches :

- **Eager (Enum):** Loads entire file into memory, performs multiple passes. Inefficient for large files.
- **Lazy (Stream):** Processes data on demand, one element at a time. Memory efficient, suitable for large or infinite streams, but sequential.
- **Concurrent (Flow):**

Elixir

```
File.stream!(path) # Produces a stream of lines
```

```
|> Flow.from_enumerable(stages: N) # Creates a flow, N concurrent stages
|> Flow.flat_map(&String.split/1) # Splits lines into words, concurrently
|> Flow.partition() # Partitions words (e.g., by hash)
|> Flow.reduce(fn -> %{} end, fn word, acc -> # Reduces words to counts, concurrently per
```

```
partition
Map.update(acc, word, 1, &(&1 + 1))
end)
|> Enum.into(%{}) # Merges results from partitions
...
```

- * `Flow.from_enumerable()`: Creates a flow from a stream, often with a specified number of parallel stages.
- * `Flow.flat_map()`: Applies a function and flattens the results, operating in parallel across stages.
- * `Flow.partition()`: Distributes data to downstream stages based on a key (by default, the element itself if used before `Flow.reduce`). This is crucial for stateful reductions like word counting, ensuring all instances of the same word are processed by the same reducer stage, preventing count inaccuracies. This is analogous to a shuffle phase.
- * `Flow.reduce()`: Performs a reduction operation within each partition.
- * `Enum.into()`: Collects and merges the results from the parallel flow stages.

When to Choose Flow vs. MQTT + GenServer ¹:

- **MQTT + GenServer**: Best for real-time, event-driven streams from distributed sources (IoT), where individual message handling and per-stream state are important. Primarily I/O-bound.
- **Flow**: Best for CPU-bound batch processing of collections, parallelizable transformations (map, filter, reduce), and high-throughput computations on finite datasets. Flow handles backpressure automatically via `GenStage`.

Combining Approaches (Hot/Cold Path) 1:

Often, these are combined:

1. **Hot Path (Real-time)**: MQTT ingests data, GenServers perform immediate processing/alerting.
2. **Buffering**: Data is also stored (e.g., database, Kafka).
3. **Cold Path (Batch Analytics)**: Flow periodically processes buffered/historical data for complex analytics, aggregations, or model training. The results from Flow can then be published back to MQTT or stored. This "Lambda Architecture" style leverages the strengths of both paradigms.

The following table summarizes key differences between GenServer-based stream processing and Elixir Flow:

Table 2: Elixir Stream Processing Approaches: GenServer vs. Flow

Aspect	MQTT + GenServer	Elixir Flow
--------	------------------	-------------

Primary Use Case	Real-time event processing, per-stream state management	Batch data transformation, CPU-intensive parallel processing of collections
Data Model	Individual, asynchronous messages/events	Collections, enumerables, streams of data
Concurrency Model	Actor-based (one GenServer per stream/entity), message passing	Pipeline parallelism, configurable number of concurrent stages
State Management	Stateful GenServers (explicit state per process)	Primarily stateless transformations; state managed within reducers/aggregators per partition
Backpressure	Manual (GenServer mailboxes can fill up) or broker-level (MQTT queues)	Automatic (handled by GenStage demand mechanism between stages)
Typical Bottleneck	Network I/O, message arrival rate	CPU processing power
Example from Snippets	MovingAverage or AbsoluteHumidity GenServers processing MQTT messages. ¹	Word counting from file using <code>File.stream!</code>

Understanding these distinctions allows for appropriate technology selection based on the specific requirements of the data processing task.

Section 7: RDF, SPARQL, and Data Modeling Exercises

This section covers the Resource Description Framework (RDF) for semantic data modeling, the SPARQL query language, and related concepts like ontologies and schema validation.

7.1. RDF Basics and Metadata Representation

RDF is a standard for data interchange on the Web, representing information as a set of triples, each consisting of a subject, a predicate, and an object.¹ For example, `(:roomA, :hasTemperature, 22.5)`.

A key challenge in RDF is representing metadata about a statement (a triple) itself—for

instance, when a particular fact was asserted or by whom. The basic RDF model does not directly support this, as a triple itself is not a resource with a URI that can be the subject of other triples.¹ This limitation is problematic for IoT scenarios where provenance (origin of data) and temporal context are vital.

Approaches like RDF reification (creating new resources to represent statements) exist but are often verbose and lack standardized semantics, making them cumbersome.¹ This difficulty in handling statement-level metadata was a significant driver for the development and adoption of mechanisms like Named Graphs.

7.2. Named Graphs for Contextual Metadata

Named Graphs extend the RDF model by allowing a set of triples (a graph) to be identified by a URI. This "graph name" or URI can then be used as a subject or object in other RDF statements, enabling the association of metadata with an entire collection of triples.¹ For example, one could state that graph_G1 (representing a set of sensor readings) wasAssertedBy :sensorX and hasTimestamp "2023-10-26T10:00:00Z".

This approach is particularly useful for:

- **Provenance:** Tracking the source or context of data.
- **Data Integration:** Managing datasets from multiple sources, where different sources might assert conflicting or context-sensitive information.
- **Modularity:** Breaking down large RDF datasets into manageable, named subsets. Named Graphs effectively transform an RDF store into a "quad store," where each piece of information is a quadruple (subject, predicate, object, graph_uri). This graph_uri acts as a context identifier. While powerful, using named graphs can increase query complexity and requires RDF stores and query engines (like SPARQL endpoints) that properly support named graph semantics and the GRAPH keyword in SPARQL.¹

7.3. SPARQL Querying

SPARQL (SPARQL Protocol and RDF Query Language) is the standard query language for RDF data. It allows for querying and manipulating data stored in RDF format. A typical SPARQL query consists of PREFIX declarations for namespace shortening, a SELECT clause specifying variables to return, and a WHERE clause containing graph patterns to match against the RDF data.¹

An example query to find rooms with temperature readings in Kelvin ¹:

Code snippet

PREFIX location: <...>

PREFIX modality: <...>

PREFIX point: <...>

PREFIX unit: <...>

PREFIX data: <...>

```
SELECT ?room ?live_temp_reading
WHERE {
  ?floor a location:Floor.
  ?floor location:contains ?room.
  ?room a location:Room.
  ?room modality:modality ?temp_mod.
  ?temp_mod a modality:Temperature.
  ?temp_sensor modality:provides ?temp_mod.
  ?temp_sensor a point:TemperatureSensor.
  ?temp_sensor unit:unit unit:Kelvin.
  ?temp_sensor data:data ?temp_data.
  ?temp_data data:live ?live_temp_reading.
}
```

Key concepts demonstrated ¹:

- **Type Constraints:** ?floor a location:Floor uses a (shorthand for rdf:type) to restrict matches to instances of a specific class.
- **Graph Pattern Matching:** Each line in the WHERE clause is a triple pattern. The query engine finds subgraphs in the data that match all these patterns. Variables (like ?room, ?temp_mod) are bound to the RDF terms they match.
- **Implicit Joins:** Multiple triple patterns sharing variables create implicit joins. For example, ?floor location:contains ?room and ?room a location:Room are joined on ?room.
- **Ontology Usage:** The query relies on the vocabulary and relationships defined in the ontology (e.g., that a TemperatureSensor provides a Temperature modality).

SPARQL's graph pattern matching is inherently flexible, designed to work with semi-structured and potentially incomplete RDF data. Features like OPTIONAL blocks allow queries to retrieve information when available but not fail if parts of a pattern are missing. This contrasts with SQL, which typically assumes a rigid schema. This flexibility is advantageous for querying heterogeneous linked data or evolving IoT datasets where not all devices report the same attributes.

7.4. Ontology Extension and Evolution

RDF ontologies (schemas defining classes, properties, and their relationships) can be dynamically extended and evolved. An exercise demonstrated adding a Building class to an existing spatial hierarchy of Floors and Rooms, reusing the location:contains property to link buildings to floors.¹

For example, adding:

:MainBuilding a location:Building.

:MainBuilding location:contains :FirstFloor.

This extends the model to Building -> Floor -> Room.¹

A SPARQL query can then verify this new structure:

Code snippet

```
SELECT ?building ?floor WHERE {  
  ?building a location:Building.  
  ?building location:contains ?floor.  
  ?floor a location:Floor.  
}
```

.¹

The **Open World Assumption (OWA)** in RDF facilitates such evolution. OWA means that the absence of a statement does not imply its falsity; it is merely unknown. Adding new classes or properties (like Building) doesn't invalidate existing data or queries that are unaware of these new constructs. Those older queries will simply not "see" the new information.¹ This contrasts with relational database schema evolution, which often requires disruptive ALTER TABLE operations and data migrations. RDF's flexibility makes it well-suited for domains where knowledge is incomplete and models evolve incrementally. However, OWA also means that inferring falsity from missing data requires additional reasoning mechanisms or closed-world assumptions explicitly added via tools like OWL or SHACL.

7.5. Schema Validation (RDFS, OWL, SHACL)

While RDF's flexibility and OWA are powerful, they also mean that RDF itself does not enforce data integrity or consistency. Any triple can be asserted. To address this, schema and validation languages are used.¹

- **RDFS (RDF Schema)**: Provides a basic vocabulary for defining classes (rdfs:Class), properties (rdf:Property), subclass relationships (rdfs:subClassOf),

sub-property relationships (rdfs:subPropertyOf), and domain/range restrictions for properties (rdfs:domain, rdfs:range).¹ It allows for simple ontological structuring.

- **OWL (Web Ontology Language):** Extends RDFS with much richer semantics for defining complex class characteristics (e.g., disjointness, unions, intersections), property characteristics (e.g., cardinality, transitivity, symmetry), and logical axioms. OWL enables automated reasoning and consistency checking.¹
- **SHACL (Shapes Constraint Language):** A W3C recommendation for validating RDF graphs against a set of conditions called "shapes." SHACL focuses on validating the structure and data types of RDF data rather than logical consistency or inference.¹ It defines constraints like mandatory properties, cardinality, data types, value ranges, and patterns.

A model checker for RDF would take an RDF graph and a set of schema rules (e.g., SHACL shapes) and report violations.¹ For instance, rules could specify that every location:Room *must* have exactly one location:area with an integer value, or that a device:TemperatureSensor *must* modality:provide a modality:Temperature.¹ Such rules can be implemented using:

- **SPARQL queries:** ASK queries or SELECT queries with FILTER NOT EXISTS can check for constraint violations.¹
- **SHACL engines:** Tools that process SHACL shapes and validate data graphs against them.¹
- **Custom scripts:** Programmatic checks within an application.

SHACL has gained traction for practical data validation because it directly addresses data quality and structural conformance, which is often more immediately relevant for application interoperability than OWL's logical reasoning. OWL is powerful for inferring new knowledge and ensuring logical soundness, while SHACL is more about ensuring data instances adhere to expected patterns.

The following table outlines the primary purpose and features of these RDF-related technologies:

Table 3: RDF Schema and Validation Technologies

Technology	Primary Purpose	Key Features	Example Use Case from	Strengths	Limitations
------------	-----------------	--------------	-----------------------	-----------	-------------

			Material		
RDFS	Basic vocabulary definition, simple class/property hierarchies	rdfs:Class, rdfs:subClassOf, rdf:Property, rdfs:domain, rdfs:range	Defining Schema.Location.Room rdfs:subClassOf Schema.Location.Location ¹	Simple to understand and use for basic ontologies.	Limited expressiveness for complex constraints (e.g., cardinality). No formal validation mechanism.
OWL	Rich ontology modeling, logical consistency checking, inference	Cardinality constraints, disjoint classes, property characteristics (transitive, symmetric), complex class definitions	Ensuring a sensor cannot be both a TemperatureSensor and a HumiditySensor if defined as disjoint. (Conceptual, not explicitly in snippets as OWL implementation)	Powerful logical reasoning, can infer new facts, ensures model consistency.	Can be complex to learn and implement; reasoning can be computationally expensive. Not primarily for data instance validation.
SHACL	RDF data graph validation against structural constraints (shapes)	Node shapes, property shapes, cardinality, datatypes, value ranges, patterns.	Defining a RoomShape requiring location:contains a device:TemperatureSensor or with minCount 1, maxCount 1. ¹	Declarative, focused on data validation, actionable reports. Good for data quality.	Not for logical inference. Expressiveness focused on graph shapes.
SPARQL (for validation)	Querying for data that violates specific,	FILTER NOT EXISTS, ASK queries, aggregation for	SELECT?room WHERE {?room a location:Room FILTER	Highly flexible for ad-hoc or custom validation	Rules must be manually written and maintained as queries;

	custom rules	constraint checks.	NOT EXISTS {?room location:name e?name } } to find rooms missing a name. ¹	rules not easily expressed in SHACL/OWL. Direct integration with RDF data.	can become complex for many rules. Not a standardized schema language.
--	--------------	--------------------	---	---	--

These technologies are not mutually exclusive and can be used in combination to build robust semantic applications.

Section 8: Integrated System Design and Analysis Exercises

This section addresses questions that require synthesizing knowledge from multiple domains covered in the exercises, such as combining ESP32 sensor data acquisition with MQTT communication and Elixir-based stream processing, and analyzing the trade-offs inherent in such integrated systems.

8.1. Designing End-to-End IoT Systems

An end-to-end IoT system typically involves several stages: data acquisition from sensors, data communication (often wireless), data processing and transformation, and finally, data storage, analysis, or actuation. The exercises provide components for each stage:

- **Data Acquisition (ESP32):** Using ADC to read analog sensors like the LMT86 temperature sensor¹ or GPIOs for digital inputs.¹ C programming is used for firmware development on the ESP32 with FreeRTOS for task management.
- **Communication (ESP32 & MQTT):** The ESP32 connects to Wi-Fi¹ and can use TCP directly¹ or MQTT (via a library like esp_mqtt_client.h) to publish sensor data to a broker.¹
- **Stream Processing (Elixir):** Elixir applications can subscribe to MQTT topics (e.g., using Tortoise), process the incoming data streams using GenServers for stateful transformations (like moving averages or absolute humidity calculation)¹, or use Flow for batch/parallel processing of larger datasets.¹
- **Data Modeling/Storage (RDF):** While not directly integrated in a full pipeline in the snippets, RDF and SPARQL could be used to model the IoT devices, their locations, sensor types, and relationships, and to store historical or processed data for semantic querying.¹

A critical, often overlooked, aspect in such pipelines is the choice of **data serialization format** at each interface (e.g., sensor-to-ESP32 if it's a digital sensor,

ESP32-to-MQTT broker, MQTT-broker-to-Elixir backend). The Elixir MQTT examples use JSON (`Jason.encode!(%{"time" =>..., "value" =>...})`).¹ JSON is human-readable and widely supported, which is advantageous for debugging and interoperability. However, it can be verbose compared to binary formats. For resource-constrained ESP32 devices and potentially low-bandwidth or costly cellular networks, more compact binary formats like Protocol Buffers or CBOR (Concise Binary Object Representation) could offer significant advantages in terms of reduced transmission size and faster parsing/serialization, albeit at the cost of human readability and possibly more complex tooling. The choice impacts parsing overhead on the ESP32, network bandwidth usage, and processing load on the backend.

8.2. Analyzing System Trade-offs

Throughout the exercises, numerous design trade-offs are encountered. Evaluating these is key to engineering effective systems. Some prominent examples include:

- **Elixir:**
 - Immutability: Enhances reliability and simplifies concurrency but can lead to more allocations if not handled carefully (e.g., with persistent data structures or by building up results efficiently).
 - if vs. case: Simplicity of if for binary choices versus the power and safety of case for pattern matching on multiple structures/values.
 - Atoms vs. Strings for Map Keys: Performance and syntax benefits of atoms versus the safety of strings for dynamic/external data due to atom table limits.
- **C Programming:**
 - Pointer Arithmetic: Offers direct memory control and potential for optimization but carries high risks of undefined behavior, memory corruption, and security vulnerabilities if not used with extreme care.¹
- **ESP32 Development:**
 - Polling vs. Interrupts for GPIO: Polling is simpler but CPU-intensive and may miss events; interrupts are efficient and responsive but more complex to manage (ISR constraints, debouncing, task communication).¹
 - atoi vs. strtol for String to Integer Conversion: atoi is simple but offers no error detection; strtol is more robust as it can indicate conversion failures.¹
- **MQTT:**
 - Quality of Service (QoS) Levels: QoS 0 (at most once) offers low latency but no delivery guarantee. QoS 1 (at least once) guarantees delivery but may result in duplicates. QoS 2 (exactly once) offers the strongest guarantee but with the highest overhead. The choice depends on the application's tolerance for message loss versus duplicates and overhead.¹

- **Stream Processing (Elixir):**

- GenServer-based processing vs. Flow: GenServers are ideal for stateful, event-driven processing of individual streams. Flow is better for stateless or batch-oriented parallel processing of collections, especially CPU-bound tasks.¹

Many design choices that seem local can have far-reaching systemic implications. For instance, using dynamically generated atoms from untrusted MQTT message payloads in Elixir for map keys could appear efficient locally but poses a system-wide denial-of-service risk if the atom table becomes exhausted. Similarly, an unsafe pointer optimization in a C module on the ESP32 might improve performance for that specific function but could destabilize the entire device if it corrupts memory. Effective system design requires a holistic view, considering how individual component choices affect overall system attributes like performance, reliability, security, and maintainability throughout its lifecycle.

8.3. Performance Evaluation and Test Harnesses

Understanding and measuring system performance is critical. The exercises touched upon a test harness (record.livemd) for instrumenting Elixir processes¹ and outlined a general performance evaluation methodology.¹

Using a test harness like record.livemd for Elixir applications typically involves¹:

1. **Wrapping Target Code:** The harness provides functions to execute the Elixir code (an "arbitrary command" or function) while collecting metrics.
2. **Metric Collection:** This can include execution duration (start/end timestamps), CPU usage, memory allocation, and message throughput.
3. **Data Output:** Results might be displayed in Livebook, saved to files (CSV, JSON), or made programmatically accessible.

A significant challenge in instrumenting concurrent systems, like those built with Elixir/OTP or FreeRTOS, is that work is often delegated to child processes or tasks. A simple harness instrumenting only the initial process might miss the resource consumption or execution time of these delegated tasks, leading to an incomplete performance picture.¹ More sophisticated harnesses would need to trace or aggregate data across related processes.

For ESP32 applications, performance evaluation might involve using ESP-IDF tools like esp_app_trace for tracing, FreeRTOS functions to get task runtime statistics (vTaskGetRunTimeStats), or measuring response times and resource usage (CPU load,

heap free) under various workloads.

Effective performance evaluation requires more than just measuring wall-clock time. It involves understanding:

- **Resource Utilization:** CPU load, memory usage (heap, stack), network bandwidth.
- **Bottlenecks:** Identifying components that limit overall system throughput or increase latency. In message-passing systems, this could be a slow GenServer whose mailbox continually grows, or a congested network link.
- **Scalability:** How the system performs as the load (e.g., number of connected devices, message rate) increases.

The following table provides a structured way to analyze various design choices encountered across the exercises, considering their impact on multiple system attributes.

Table 4: System Design Trade-off Analysis Matrix

Design Choice / Scenario	Performance (Latency)	Performance (Throughput)	CPU Usage	Memory Usage	Reliability	Scalability	Development Complexity	Security Implications
Polling vs. Interrupt (ESP32 GPIO Input) 1	Interrupt: Lower (faster response)	Interrupt: Higher (can handle more events)	Polling: Higher (constant checking)	Similar	Interrupt: Lower if debouncing is poor; Polling: Can miss events	Interrupt: Better for many events	Polling: Lower	N/A directly, but missed critical events could be a safety issue.
TCP vs. MQTT (ESP32 Data)	TCP: Potentially lower for	TCP: Depends on connection	Similar for basic send/receive.	MQTT client adds memory	TCP: Connection-oriented, Reliability	MQTT: Better for many-to-many	TCP: Lower-level socket programming	TCP: Direct exposure of server.

Flow vs. GenServer Pipeline (Streaming) ¹	Higher for batch setup, lower per-item if highly parallel. GenServer: Lower for first item, consistent per item.	Higher for large, CPU-bound batches. GenServer: Limited by single-process speed per stream.	Higher (utilizes multi-core). GenServer: Lower per instance.	Can be higher due to batching/buffering. GenServer: Lower per instance state.	Good with GenStage backpressure. GenServer: Mailbox can overflow if producer is faster.	Scales with cores for batch. GenServer: Scales by adding more processes for more streams.	Higher-level API, can be complex for state. GenServer: More boilerplate for simple transforms.	
---	--	---	--	---	---	---	--	--

This matrix encourages a multi-dimensional analysis of design decisions, reflecting the complexities faced in real-world system development.

Section 9: Conclusion

The exercises explored in this guide span a wide array of technologies and programming paradigms, from the functional elegance of Elixir and its powerful OTP framework for concurrency, to the low-level control of C for systems programming and algorithm implementation, and finally to the practicalities of embedded development on the ESP32 with FreeRTOS. Key themes emerge across these diverse areas:

1. **Concurrency Management:** Whether through Elixir's lightweight processes, GenServers, and Tasks, or FreeRTOS tasks on the ESP32, managing concurrent operations efficiently and safely is paramount. The choice of mechanisms—polling, interrupts, message queues, asynchronous tasks, or actor-based servers—depends heavily on the specific requirements for responsiveness, resource utilization, and system complexity.
2. **Abstraction and Modularity:** Encapsulating functionality into reusable components (Elixir modules, C functions, ESP-IDF components like `hl_wifi`) is crucial for managing complexity, improving maintainability, and enabling robust system design.
3. **Trade-offs in Design:** Nearly every technical decision involves trade-offs—simplicity versus robustness, performance versus resource

consumption, flexibility versus safety. Recognizing and evaluating these trade-offs (e.g., if vs. case in Elixir, pointer arithmetic in C, polling vs. interrupts on ESP32, MQTT QoS levels) is a hallmark of proficient engineering.

4. **Data Representation and Transformation:** The way data is structured (Elixir maps vs. structs, RDF triples), serialized (JSON vs. binary formats), and transformed (Elixir's Enum and Stream vs. Flow, ADC calibration, SPARQL queries) significantly impacts system performance, interoperability, and correctness.
5. **Event-Driven Architecture:** Many modern systems, especially in IoT, are event-driven. The ESP-IDF event loop, Elixir's message passing, and MQTT's publish-subscribe model all exemplify this paradigm, enabling responsive and decoupled systems.

A deep understanding of these core concepts, coupled with the practical experience gained from implementing and analyzing the exercises, provides a solid foundation for tackling complex software and IoT system design challenges. The ability to select appropriate tools and patterns, reason about their implications, and integrate them effectively is essential for developing solutions that are not only functional but also efficient, scalable, and reliable.